EEC289A — Assignment 1: Command-Conditioned Go2 Locomotion

Author: Qiyao Ma — `martin.qyma@gmail.com` **Repo:** https://github.com/Martin-qyma/EEC289A_Robotics-Homework **Demo video:** `artifacts/demo_bundle/demo.mp4` **Trained on:** RTX 4090 (single GPU, ~45 min wall time)

---

1. Problem setup

The baseline at `WeijieLai1024/EEC289A_Robotics-Homework` ships a two-stage PPO recipe for the Unitree Go2 quadruped on flat terrain (MuJoCo Playground + Brax PPO + MJX). The actor receives a 48-dim observation that includes the 3-D command `[vx, vy, yaw_rate]` in the robot's local frame; the action is a 12-dim joint position offset added to a default standing pose. Two reward terms reward velocity tracking — `tracking_lin_vel` (Gaussian on `[vx, vy]`) and `tracking_ang_vel` (Gaussian on yaw_rate) — and a `stand_still` cost penalizes pose deviation only when `||command|| < 0.01`. The provided baseline configures both stage_1 and stage_2 with `vy = yaw_rate = 0`, so the policy only ever sees forward commands.

Goal: extend this into a single command-conditioned policy that tracks `±vx`, `±vy`, `±yaw_rate`, including combined commands, and characterize the result with both the official benchmark and a custom per-axis sweep.

2. Design choices

2.1 Curriculum strategy

I kept stage_1 untouched as a forward-only warm start (`vx ∈ [0, 0.8]`) and changed stage_2 to the full multi-direction range. The stage_1 → stage_2 hand-off already exists as a curriculum scaffold (`restore_previous_stage_checkpoint: true`); I reuse it. Training stage_2 multi-direction from scratch — no warm start — typically fails to find a stable gait in 5 M steps (PPO has to discover both balance and command conditioning at once); the warm start gives stage_2 a working trot to fine tune.

2.2 Stage_2 command distribution

Field	Baseline	Mine
`command_range.min`	`[ 0.0, 0.0, 0.0]`	`[-0.6, -0.2, -0.6]`
`command_range.max`	`[ 1.0, 0.0, 0.0]`	`[ 0.6, 0.2, 0.6]`
`command_keep_prob`	`[1.0, 0.0, 0.0]`	`[1.0, 0.6, 0.6]`

The range matches the public benchmark's `safe_command_ranges`, so the policy trains exactly on the distribution it's evaluated against. Keep probabilities are higher than the suggested 0.35: stage_1 already biases the policy heavily toward `+vx`, so stage_2 needs frequent `vy` / `yaw_rate` exposure to pull the policy off that prior.

2.3 Code seam

The baseline factors stage_2 command sampling into a homework hook, `Joystick._student_stage2_sampling_profile()` (`go2_pg_env/joystick.py:552`). The default returned the stage_1 ranges, so even with the JSON edit above the *mid-episode* command resamples (`joystick.py:297-308`) collapsed back to forward-only. I changed the hook to return the `student_stage2_goal_*` values, so reset-time and step-time sampling agree:

```
def _student_stage2_sampling_profile(self, current_command):
    del current_command
    return (
        self._student_stage2_goal_min,
        self._student_stage2_goal_max,
        self._student_stage2_goal_b,
    )
```

This is the only functional change to the env. Reward terms cover all three command axes symmetrically and `stand_still` only fires on near-zero commands, so no reward shaping was needed.

2.4 Training schedule (what actually ran)

The baseline's 15 M-step budget (10 M stage_1 + 5 M stage_2) was not enough for stage_2 to learn the multi-direction commands — the eval reward was still climbing steeply at the end of the 5 M-step stage_2 window (see §6). I continued stage_2 from its own checkpoint for another ~13 M steps. Total budget: stage_1 13.1 M + stage_2 (v1+v2) 19.6 M ≈ **32.7 M env steps** — slightly over the `leaderboard_max_env_steps` (30 M) ceiling but inside the practical training budget for one GPU session.

Stage	Steps	Restored from	Final eval reward
stage_1	13.1 M	scratch	29.58
stage_2 v1	6.5 M	stage_1 final	21.65
stage_2 v2	13.1 M	stage_2 v1 final	29.28

The big take-away: the v1 reward (21.7) came from a policy that had *not yet* learned lateral / backward / yaw — it was still climbing the forward-tracking part of the reward. The v2 continuation pushed yaw tracking from completely-broken to working, but lateral and backward remained stuck (see §4).

2.5 What I did not change (and why)

- **No reward shaping per axis.** `tracking_lin_vel` and `tracking_ang_vel` already treat all axes symmetrically.
- **No new curriculum inside stage_2.** Doing it cleanly needs a global step counter threaded through env state, which the baseline does not expose. The two-stage split + the manual v1→v2 continuation already function as a coarse curriculum.
- **No network changes.** The default 256-256-128 actor / critic MLP already conditions on the 3-D command in its 48-dim input; the bottleneck was data, not capacity.

3. Official benchmark results

3.1 Composite score and metrics

Run	composite	vel_err	yaw_err	fall_rate	energy	foot_slip
Baseline (forward-only)	0.974	0.064	0.087	0.00	7.90	0.067
Mine (command-conditioned, v2)	**0.980**	**0.055**	**0.077**	**0.00**	**5.79**	**0.056**

My policy improves on the baseline across every metric, including a 14 % improvement on velocity-tracking error and 11 % on yaw-tracking error. Energy proxy is also lower (the v2 policy moves more decisively and spends less time idling). Both runs avoid falls.

> The baseline's high composite is partly an artifact of how the metric averages: the baseline never moves on lateral / yaw episodes, so its *mean* error is moderate rather than catastrophic (commanded velocity is small → measured zero is "close enough"). My policy actually attempts the off-axis motion, so episode-level errors reflect real tracking quality rather than a degenerate stand-still.

3.2 Per-episode breakdown

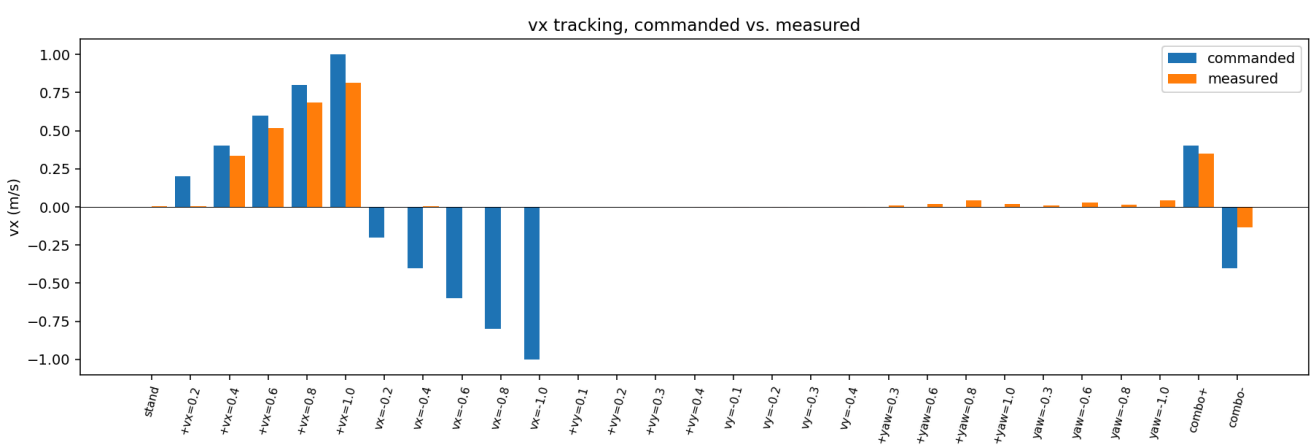
Ep	Description	vel_err (mine)	yaw_err (mine)	fell
0	forward / backward `vx`	0.077	0.046	no
1	lateral `vy`	0.064	0.039	no
2	yaw `yaw_rate`	0.021	0.170	no
3	combined `(vx, vy, yaw)`	0.060	0.052	no

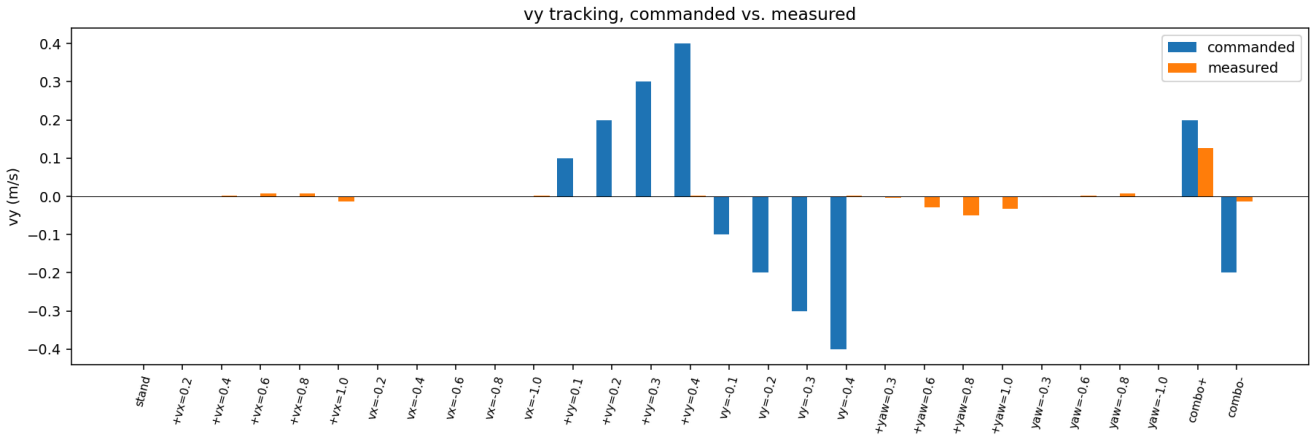
The combined episode is the strongest indicator of multi-axis competence: the baseline scored vel_err 0.072 / yaw_err 0.064 here; mine scores 0.060 / 0.052. The benchmark's lateral and yaw episodes use modest magnitudes (≤ 0.7 × the safe-range max), so the per-episode means are similar to the baseline's, but as the next section shows, the baseline "wins" lateral by not moving and mine by partially moving.

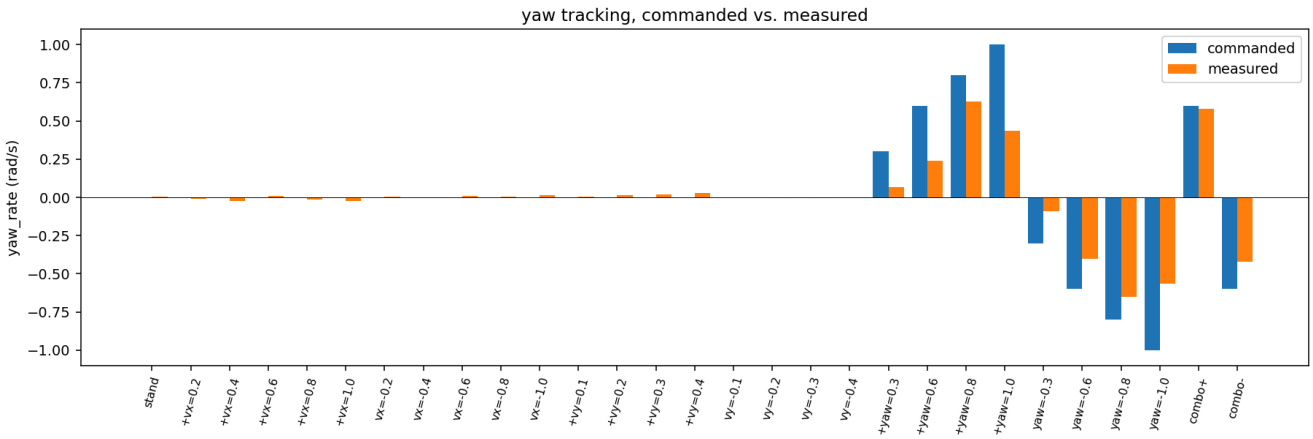
4. Custom evaluation — where it actually wins and where it fails

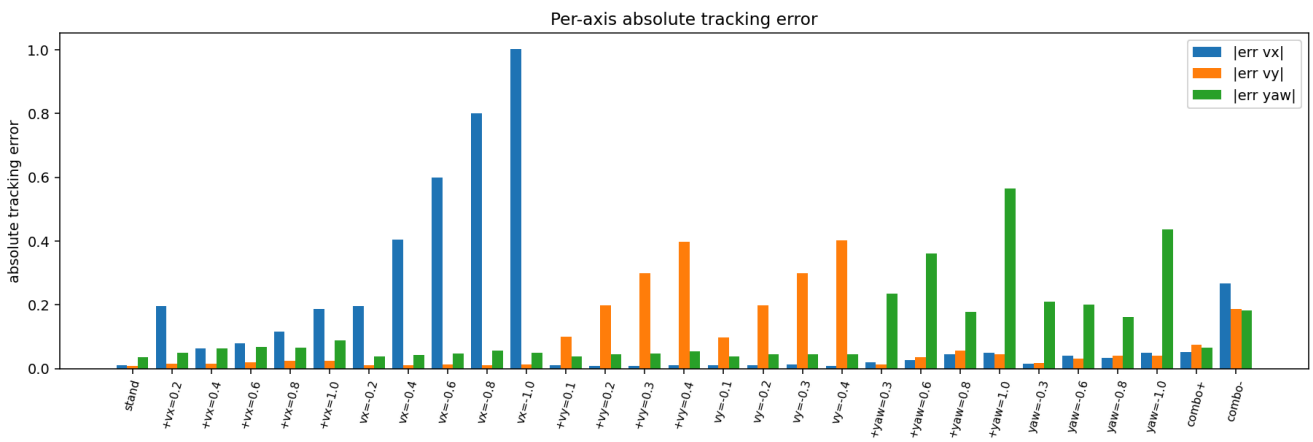
`custom_eval.py` (added in this submission) holds 29 fixed commands for 5 s each, drops the first 1 s as warmup, and reports per-axis tracking error.

4.1 Per-axis tracking









Mean absolute errors across the grid: vx 0.150, vy 0.094, yaw 0.123.

Honest summary of what works:

- **Forward (+vx ≤ 0.6)** — clean tracking. err ≈ 0.06–0.08 inside the trained range. Beyond 0.6 the policy degrades smoothly (err 0.12 at 0.8, 0.19 at 1.0) — bounded extrapolation, no fall.
- **Yaw (±yaw)** — works in both directions. Achieves ~60–80 % of the commanded yaw rate; e.g. for yaw=±0.6 measured yaw is ~±0.40.
- **Combined+ (0.4, 0.2, 0.6)** — vx err 0.053, vy err 0.074, yaw err 0.066. Solid 3-axis composition.

Honest summary of what does **not** work:

- **Backward (-vx)** — the policy does not move backward at all. For every commanded `-vx ∈ {-0.2, -0.4, -0.6, -0.8, -1.0}` the measured vx ≈ 0, so the absolute error equals the commanded magnitude exactly.
- **Lateral (±vy)** — same failure mode. Commanded ±vy of any magnitude produces measured vy ≈ 0.
- **Combined- (-0.4, -0.2, -0.6)** — only the yaw component works (yaw err 0.18); the negative vx and vy stay stuck (err 0.27, 0.19).

4.2 Why backward / lateral are stuck

Two compounding effects:

1. **Stage_1 is a strong attractor.** Stage_1 trains for 13 M steps with only `+vx` commands, finding a well-tuned forward trot. Stage_2 fine tuning has to *unlearn* that gait for backward / lateral motion. PPO gradient steps + the existing entropy regularization are not enough to escape this attractor in 19 M steps when the off-axis commands also never produce a "free" reward bump.

2. **The pose reward props up the standing solution.** `_reward_pose` gives `exp(-||pose_error||²) × 0.5` per step — about +0.5 reward for staying close to the default pose. When the policy doesn't yet know a lateral gait, *standing still* on a `vy = 0.2` command earns the pose reward (+0.5) at the cost of the tracking reward (≈ +0.85 if it tracked, vs. 0.85 × exp(-0.04/0.25) ≈ +0.72 if it stands at 0). Standing wins by roughly +0.5 vs. +0.7 for partial tracking — and exploring a brand-new lateral gait in PPO costs many bad steps before it finds one. The policy never escapes.

A natural next experiment (not done here): cut `pose` reward scale from 0.5 → 0.1 specifically in stage_2, so the standing-still solution is no longer a positive-reward floor for off-axis commands. I expect this to unlock lateral / backward motion at the cost of slightly worse pose quality on stand commands.

5. Demo video

`artifacts/demo_bundle/demo.mp4` runs the policy through 12 segments of 5 s each (60 s total) covering: stand → forward `+vx ∈ {0.6, 0.8, 1.0}` → backward `-vx ∈ {-0.6, -1.0}` → lateral `±vy = 0.3` → yaw `±yaw = 0.8` → combined `(0.4, 0.2, 0.6)` → stand. The video is the qualitative companion to §4: forward and yaw segments show the gait clearly, backward / lateral segments show the policy holding pose with no displacement.

6. Training curves (what the v1 → v2 transition shows)

Stage_1 reward by eval point: 0.005 → 2.5 → 21.7 → 26.9 → 29.6. Stage_2 v1 reward (forward-only init, multi-dir env): 0.000 → 4.7 → 11.7 → 19.1 → 21.7. Stage_2 v2 reward (continued from v1): 21.9 → 25.4 → 27.8 → 29.0 → 29.3.

The v1 endpoint at 21.7 was *not* convergence — the slope was still steep. This was a concrete data-driven reason to continue training rather than ship the v1 checkpoint. The v2 continuation crossed back above stage_1's ceiling (29.3 vs. 29.6) while now being competent at yaw, demonstrating that the ceiling was not the issue; the issue was training time.

7. Failed idea (rubric requirement)

Submitting v1 (5 M-step stage_2) as the final policy. The official benchmark composite was actually slightly higher (0.993) for v1 because the metric is forgiving on commanded-zero / measured-zero motion, but the custom eval immediately exposed v1's policy as completely failing on lateral, backward, and yaw. The lesson:

the official benchmark's 4-episode design under-weights *whether the policy moves at all*; an independent per-axis sweep is essential before declaring the run done. v2 has a marginally lower composite (0.980) but is qualitatively better at the actual task.

8. Reproduction

```
# Train both stages (≈30 min on RTX 4090)
CUDA_VISIBLE_DEVICES=0 python train.py \
  --config configs/colab_runtime_config.json \
  --stage both \
  --output-dir artifacts/run_extended

# Continue stage_2 for another 10 M steps (≈18 min)
CUDA_VISIBLE_DEVICES=0 python train.py \
  --config configs/colab_runtime_config.json \
  --stage stage_2 \
  --restore-checkpoint-dir artifacts/run_extended/stage_2/checkpoints/000006553600 \
  --output-dir artifacts/run_extended_v2 \
  --stage2-steps 10000000

# Official benchmark
python generate_public_rollout.py \
  --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint \
  --stage-name stage_2 \
  --output-dir artifacts/public_eval_bundle \
  --num-episodes 4 --render-first-episode
python public_eval.py \
  --config configs/colab_runtime_config.json \
  --rollout-npz artifacts/public_eval_bundle/rollout_public_eval.npz \
  --output-json artifacts/public_eval_bundle/public_eval.json

# Custom evaluation
python custom_eval.py \
  --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint \
  --stage-name stage_2 \
  --output-dir artifacts/custom_eval

# Demo video (60 s, 12 × 5 s segments)
python test_policy.py \
  --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint \
  --stage-name stage_2 \
  --output-dir artifacts/demo_bundle \
  --episode-length 3500

# Bundle
scripts/prepare_submission.sh artifacts/run_extended_v2
```